Adaptive Thresholding

John Wensink

CSC515 Foundations of Computer Vision

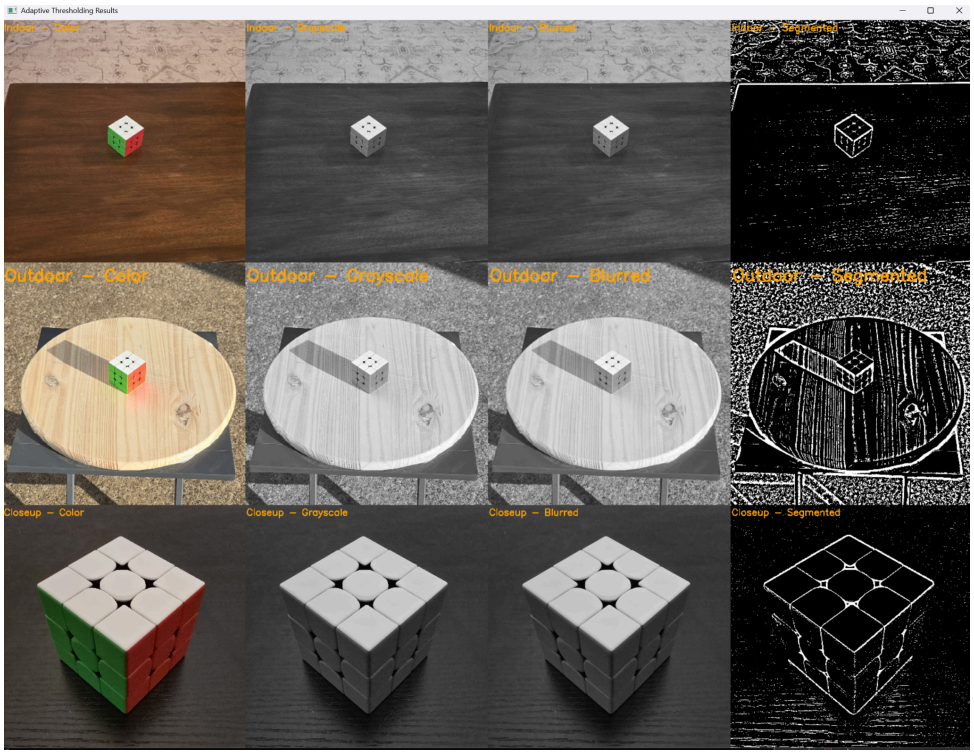
Colorado State University – Global Campus

Dr. Bingdong Li

April, 27, 2025

Adaptive Thresholding

The goal of this week's project was to apply an adaptive thresholding technique to segment images taken in different environments, including an indoor scene, an outdoor scene, and a close-up of a single object (OpenCV, n.d.) I found a Rubik's Cube that I thought would be a good subject for these photographs, because of its high contrast colors, clean edges, and small size, expecting that the outlines of each individual block on the cube would show up easily when using adaptive thresholding. I took three separate photos, one inside on a brown wooden table, one outside on a small patio table under direct morning sunlight, and one very close-up on a black wooden desk. Each of these environments ended up presenting more challenges for segmentation than I expected, mostly due to the textures of the surfaces the cube was on, as well as variations in lighting.

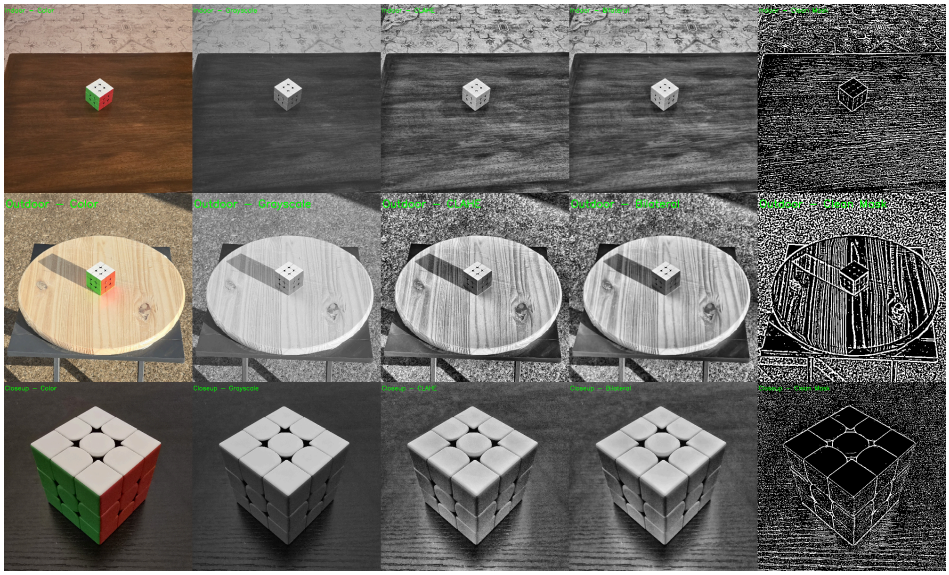


The images were loaded into Python using OpenCV's imread() function and stored in a dictionary that mapped scene names ("Indoor", "Outdoor", and "Closeup") to their file paths. Using a dictionary allowed me to efficiently loop through the images without having to hardcode individual function calls. I also initialized an empty list to store the processed versions of the images as they moved through each stage of the pipeline. Keeping the code organized this way made it easier to make global updates, troubleshoot issues, and keep the final display grid clean and predictable.

Each image went through several preprocessing steps inside a function called process_image. First, I converted the color image to grayscale. Since thresholding works best based on intensity values rather than color information, grayscale was a necessary simplification step. I then applied a Gaussian blur with a 5x5 kernel size to reduce noise and small pixel-level variations that could otherwise interfere with the adaptive thresholding process. After smoothing the image, I applied adaptive mean thresholding where the threshold value for each pixel is based on its local neighborhood, minus a constant value. For this project, I used a block size of 35 pixels and a subtraction constant of 8 to achieve what I found to be the best results. These values were chosen through experimentation, as they provided the best balance between object separation and noise control across all three scenes.

I also experimented with the thresholding mode (Rosebrook, 2021), finding an inverted binary technique to give the best results. This setting inverts the typical behavior. Pixels that are brighter than the local threshold will become black, and pixels that are darker than the threshold will become white. In my images, especially under indoor lighting, the cube's colored faces tended to be darker than the background surfaces, so inversion made the object stand out more clearly against the background. I tested other thresholding modes, such as truncation and to-zero

thresholding, but they tended to preserve too much background information and left the final image noisy, even after experimenting with histogram equalization (CLAHE) preprocessing steps (GeeksforGeeks, 2023.)



Once the thresholding was complete, I created a dictionary to organize the four stages of each image (color, grayscale, blurred, and segmented), a fifth column for CLAHE was also included in the to-zero and truncation methods, but was ultimately removed for the final submission. Before adding any labels to the images, I converted the grayscale outputs back to BGR color space. This was because OpenCV's text-drawing function, putText (Sharma, 2021), expects a three-channel image when applying colored text, which was needed to stand out against the grayscale images. I set the font scale to a relatively large size of three and the thickness to 10 to make sure the labels stand out at a glance. While the labels themselves used the same font size and thickness, I noticed during testing that the visual size of the labels appeared slightly different depending on the label text and background contrast. For example, the

outdoor label appeared slightly larger than the indoor label because of the wider letter shapes and higher background brightness, but the underlying font settings remained consistent.

After labeling, I resized each image to 400x400 pixels to ensure uniformity when stacking the images together. For each scene, the four images were stacked horizontally to form a single row. The three scene rows were then stacked vertically to produce the final grid. This structure allowed for quick side-by-side comparison of the processing steps across different environments. I displayed the final result in a single OpenCV window using imshow() and kept the window open until a key press allowed it to close.

Overall, this project provided a good opportunity to build an end-to-end image preprocessing and segmentation pipeline. It reinforced the importance of adaptive techniques for handling real-world variability, as well as the value of clean code organization and presentation. Managing different types of images consistently, applying labels clearly, and presenting the results in a readable format all made a noticeable difference in the quality of the final output. The main challenge was adjusting block size and constant values that worked well across varying lighting and background conditions. Experimentation and visual feedback made it manageable. In the future, it would be interesting to explore adaptive thresholding methods that apply morphological operations to refine the segmented output further. For this assignment, the current approach produced reliable and visually clean results across the different scenes.

References

GeeksforGeeks. (2023, May 19). CLAHE Histogram Equalization – OpenCV. GeeksforGeeks. Retrieved April 20, 2025, from https://www.geeksforgeeks.org/clahe-histogram-eqalization-opencv/

OpenCV. (n.d.). Adaptive Thresholding. Retrieved April 27, 2025, from https://docs.opencv.org/4.x/d7/d4d/tutorial_py_thresholding.html

Rosebrock, A. (2021, April 28). OpenCV thresholding: cv2.threshold. PyImageSearch. Retrieved April 27, 2025, from https://pyimagesearch.com/2021/04/28/opencv-thresholding-cv2-threshold/

Sharma, Y. (2021, June 22). OpenCV putText() – Writing text on images. AskPython. Retrieved April 27, 2025, from https://www.askpython.com/python-modules/opencv-puttext